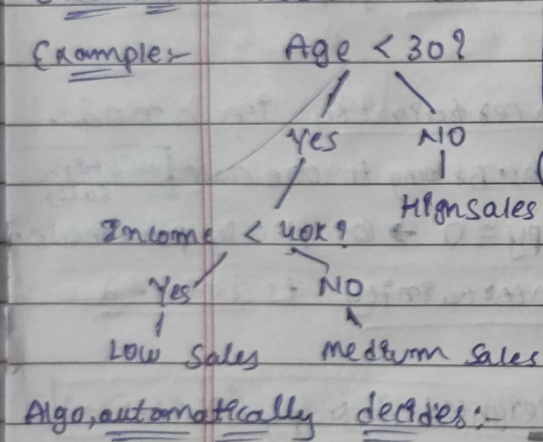
for tasks such as Classification, Regression, Clustering & Dimensionality

21-09-2026 (Monday)

(3) (4) Decision Tree algorithm:- Supervised ML algorithm that makes predictions by learning a sequence of decision rules from the features.

Structure:- Root → Internal Nodes → Branches → Leaf.

Example:-



Algo, automatically decides:-
① which feature to use ② where to split the feature ③ which split should come first?
④ when to stop splitting.

① Root Node:- Top most node of the tree. Represent entire data set before splitting. Age<30 → Root Node.

② Internal Node:- Represents a decision, based on a feature. Eg:- Income > 50,000? [Each internal node divides data further.

③ Branch:- Represent outcome of a decision.

④ Leaf Node:- Final node of the tree. contains final prediction. Eg:- Classification - High sales, Low sales
Regression - ₹5,20,000.

Age < 30
Yes
NO

classmate
Date
Page

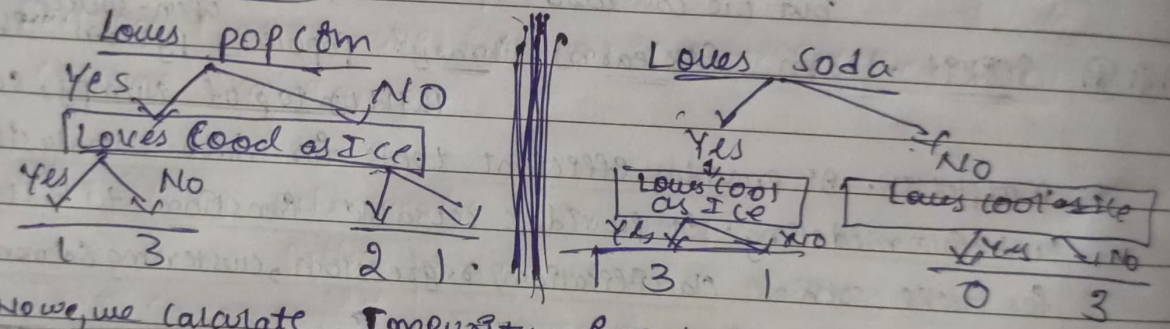
Eg:

Loves popcorn	Loves soda	Age	Loves cool as Ice.
Yes	Yes	7	No
Yes	No	12	No
No	Yes	18	Yes
No	Yes	35	Yes
Yes	Yes	38	Yes
Yes	No	50	No
No	No	83	No

How to select ~~branch~~ ?

We take each column (2) verify with target ~~Yes~~ column (1) Checks which better clarity means less impurity.

For ~~the~~ categorical columns:



→ Now, we calculate Impurity ~~from~~ both columns.

(*)(*) Splitting Criteria :- For Decision tree classification, commonly used criteria are : (3)

→ (1.) Gini Impurity :- Measures how impure a node is.

$$Gini = 1 - \sum p_i^2$$ → proportion of samples belong to class

(*) Note: Lower Gini = purer node (Gini = 0 → perfectly pure node). If a node contains only one class : 100% High sales, Then Gini = 0

→ (2.) Entropy :- Measures randomness (&) uncertainty in a node.

$$Entropy = -\sum p_i \log_2(p_i)$$

(*) Note: Lower entropy = less uncertainity

If all obs. belong to one class [100% High sales] Entropy = 0 → Completely pure node

→ (3.) Information Gain :- measures how much uncertainity is reduced after splitting.

$$Information\ Gain = Entropy(Parent) - Weighted\ Entropy(Children)$$

(*) Note: Decision Tree prefers, Higher Information Gain.

what is Pruning, Bagging (&) Boosting in Random Forest
How decision tree works for Regression:

For Numerical column; we will ① Sort the values
② Find average between each 2 values
③ For each average value, we calculate Gini impurity.
④ Consider lowest impurity value with other categorical Gini impurity, which is lowest, can be consider as Root node.

⋆⋆⋆ For Decision Tree algorithm, we don't need to perform feature scaling.

⋆⋆⋆ Splitting:- Dividing a dataset into smaller group based on feature.

Age	Income	Sales
22	20K	Low
25	25K	Low
35	60K	High
40	70K	High

The tree may find : Age < 30 ?
Then: Age < 30 + Low Sales | Age >= 30 + High Sales.

A good split create groups that are more homogenous / pure.

Decision Tree - Classification

→ Target is category / class.
Eg. Spam / Not spam, Pass / Fail

Gini	Entrophy
measure impurity	measure uncertainity
Lower is better	Lower is better
used by criterion = "gini"	criterion = "entropy"
Faster computationally	slightly slower.

Income >= 50K ?
Yes → High Sales
No → Low Sales
→ Leaf contains predicted class.

⋆⋆ → Decision Tree for Regression:- To predict continues numerical variables.

Eg: House Price, Salary, Sales, Revenue

Area < 1500 ?
Yes → Bed room < 3 ?
Yes → ₹40 Lakh.
No → ₹55 Lakh.
No → ₹80 Lakhs

At the leaf node, the prediction is generally the average target value of the training samples in that leaf.

Suppose a leaf contains:- ₹40L, ₹42L, ₹38L, ₹44L.

prediction = $\frac{40+42+38+44}{4} = 41$

So, the tree predicts ₹41 Lakhs

(*) Splitting Criterion for Regression:-

For regression, we don't use Gini (&) Entropy.
Common criteria include: Squared Error / MSE
absolute Error, Friedman MSE
The tree searches for splits, reduce prediction error within resulting groups.

(*) Overfitting:- When a model learns training data too closely, including noise (&) random patterns.

Training → Very good	Training accuracy = 100%
Testing → Poor	Testing " = 75%

Note:- Decision tree is overfit, as it keeps splitting until it creates extreme small groups.

(*) under fitting:- when model is too simple to learn important pattern in data (lack of training data)

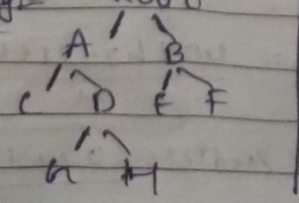
Training → ~~poor~~ low	Training Accuracy = 65% } Model has not
Testing → Poor	Testing " = 63% } learned enough

Diff

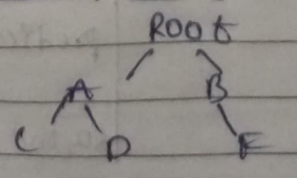
Overfitting	under fitting	Classification	Regression
(1) model too complex	model too simple.	Predict categories	Predict Numerical value
(2) learn noise	misses imp. patterns.	Eg: (1) High / Low	Eg: ₹50,000
(3) Training → High	training → low	(2) Gini	MSE / Square Error
Testing ~~performs~~ → Poor	Testing → poor	(3) Entropy	(4) Absolute Error.
		(4) Leaf give class Prediction	(5) Leaf give always numerical value.

(*)(*)(*) Pruning:- Removing unnecessary branches / nodes from a decision tree to reduce complexity (&) prevent overfitting.

Before pruning:-



After pruning:-



Some unnecessary branches are removed.

Without pruning :- Tree becomes deep, Learns noise, overfitting.

With " :- Remove unnecessary branches, simple tree, Less complex, Reduce overfitting.

Types :- ① Pre-Pruning :- Stops the tree before it becomes too complex.

Common parameters: max_depth,
min_samples_split,
min_samples_leaf,
max_leaf_nodes

Eg :- DecisionTreeClassifier(max_depth=5)
prevents tree growing beyond depth 5.

② post-pruning :- Remove unnecessary branches in complex tree.

Eg :- ccp_alpha
(cost complexity pruning)

---

⑭ Random Forest :- It is an ensemble ML algorithm, that combines multiple Decision Trees to make a stronger prediction.

Instead of : one decision Tree.

We create : Tree 1, Tree 2, Tree 3 ---- Tree 100 (combine predictions).

For classification : majority voting

regression : Average of predictions.

→ Random Forest - Forest of many Decision Trees.

Randomness sources ① Random Samples of training data.

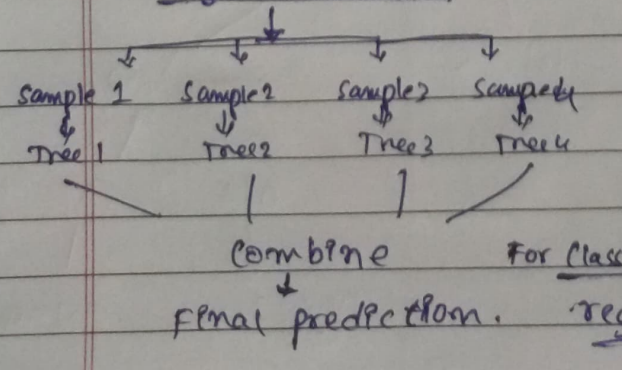
② " Subset of features.

→ ⑭ Bagging = Bootstrap Aggregating.

① create multiple bootstrap samples from the training data.

② Train a separate model on each sample.

③ Combine their predictions.



Note :- Random Forest uses Bagging, not boosting.

→ It ensembles Decision trees using bootstrap Sampling + random feature selection + aggregation.

For Classification: Voting
regression: Average.

→ ⑧ Boosting :- Another ensemble technique, ~~to~~ built models sequentially

Tree 1
↓
Find Errors
↓
Tree 2 focuses on difficult cases
↓
Find remaining errors
↓
Tree 3
↓
Combine

Bagging	Boosting
① models can be trained independently.	① models are built sequentially.
② uses bootstrap samples	② uses previous errors.
Eg: Random Forest	Eg: Gradient boosting, Ada Boost, XG Boost etc.

Bagging → Built many models independently (Σ) Aggregate.
Boosting → " models one after another, improve previous mistakes